Install and Run guide-TerraVista

1 — Prerequisites

- Python 3.10+ installed and on PATH
- pip (latest)
- Git (optional)
- (Optional) GPU + CUDA + NVIDIA drivers for faster training/inference
- Ports: app uses 5000 by default (changeable in app.run())

---

2 — Project structure (expected)

Place files in one folder:

project-root/

├– app.py

├– final_frontend.html

├– requirements.txt

├– runs/ # created by training or by you (weights go here)

└– data/ # dataset used for training (optional)

---

3 — Recommended virtual environment

Linux / macOS

python3 -m venv .venv

source .venv/bin/activate

python -m pip install --upgrade pip

Windows (cmd)

python -m venv .venv

.venv\Scripts\activate

python -m pip install --upgrade pip

Windows (PowerShell)

python -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

---

4 — requirements.txt (copy/paste)

Create requirements.txt in repo root with:

flask>=2.0

flask-cors

ultralytics>=8.0

pillow

numpy

google-generative-ai

gunicorn # only if you plan to run Gunicorn

Note: google-generative-ai is the Python package that provides google.generativeai import (used in your app.py). If that package name changes in the future, install whatever the official Google Generative AI client package is.

Install the requirements:

pip install -r requirements.txt

---

5 — PyTorch / GPU notes

- **CPU only**: ultralytics will work but inference/training will be slow.
- **GPU (recommended for training/inference)**: Install a PyTorch build compatible with your CUDA version. Example (Linux) — replace <cu_version> with your CUDA version (see PyTorch site for exact command):

example (replace with the correct index URL for your CUDA)

pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

Then reinstall ultralytics:

pip install ultralytics

If you are unsure, install CPU PyTorch first or visit the official PyTorch install page to copy the exact command for your environment.

---

6 — Environment variables (Gemini API key)

DO NOT hardcode API keys. Set them as environment variables.

Linux / macOS

export GEMINI_API_KEY="your_real_key_here"

Windows (cmd)

set GEMINI_API_KEY=your_real_key_here

Windows PowerShell

$env:GEMINI_API_KEY="your_real_key_here"

In app.py you should change:

GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")

genai.configure(api_key=GEMINI_API_KEY)

(If not already reading from env, modify the script to avoid committing secrets.)

---

7 — Training (optional)

If you want to train a classifier (your second block in app.py):

1. Prepare dataset structure (example for classification expected by ultralytics):

data/

├─ train/

├─ val/

└─ test/

2. Edit config variables at top of training section in app.py (e.g., RAW_DIR, EPOCHS, BATCH_SIZE, RUN_NAME).
3. Run training (from terminal):

python app.py

The training part of your app.py runs in if __name__ == "__main__":. If you prefer separate scripts, extract training code to train.py and run python train.py.

4. After training, weights are saved under runs/<name>/weights/best.pt (or last.pt). The loader in app.py will pick the latest run automatically.

---

8 — Run the web app (development)

Ensure .venv activated and GEMINI_API_KEY set.

python app.py

This prints:

🚀 Starting server at http://localhost:5000

And auto-opens browser (your file has a Timer that opens the browser after a second). The app serves final_frontend.html at root.

---

9 — Quick API test (curl)

Replace image.jpg with a test image.

curl -X POST -F "image=@image.jpg" http://127.0.0.1:5000/predict

Expected JSON:

{

"success": true,

"disease": "apple_scab",

"confidence": "93.2",

"advice": "..."

}

---

10 — Production suggestions

Gunicorn (Linux)

Install gunicorn (already in requirements.txt), then:

from project-root

GEMINI_API_KEY="..." gunicorn -w 4 -b 0.0.0.0:5000 app:app

Wrap gunicorn with systemd or run behind Nginx as a reverse proxy for SSL and static serving.

systemd unit example

Create /etc/systemd/system/plant-detect.service:

[Unit]

Description=Plant Detect Flask App

After=network.target

[Service]

User=ubuntu

Group=www-data

WorkingDirectory=/path/to/project-root

Environment=GEMINI_API_KEY=your_real_key_here

ExecStart=/path/to/project-root/.venv/bin/gunicorn -w 3 -b 127.0.0.1:5000 app:app

[Install]

WantedBy=multi-user.target

Then:

sudo systemctl daemon-reload

sudo systemctl start plant-detect

sudo systemctl enable plant-detect

Use Nginx to proxy 127.0.0.1:5000 and add TLS.

---

11 — Docker (optional)

Dockerfile (minimal) — put in repo root:

```
FROM python:3.10-slim

WORKDIR /app

# system deps for image libs (opencv / pillow)
RUN apt-get update && apt-get install -y \
    build-essential \
    libgl1 \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

ENV FLASK_ENV=production
ENV PYTHONUNBUFFERED=1

EXPOSE 5000
CMD ["python", "app.py"]
```

Build & run:

docker build -t plant-detect:latest .

docker run -e GEMINI_API_KEY="your_key" -p 5000:5000 plant-detect:latest

GPU Docker: pick an appropriate base image (nvidia/cuda) and use nvidia runtime if you need GPU._

---

12 — Common troubleshooting & fixes

- **ModuleNotFoundError: ultralytics**
  pip install ultralytics (ensure venv activated).
- **Model load error / runs/ folder missing**
  Make sure runs/ exists and contains a run folder with weights/best.pt or weights/last.pt. You can place a pre-trained best.pt under runs/some_run/weights/best.pt.
- **Gemini errors / API key issues**
  - Ensure GEMINI_API_KEY is set in the environment where the server runs.
  - Check that the package google-generative-ai is installed and importable.
  - Catch exceptions in app.py and return a friendly message (you already have fallback).
- **Slow performance (CPU)**
  Use a machine with GPU or run smaller image sizes during inference. Pre-warm the model by keeping it loaded (your app does this).
- **CORS issues**
  Ensure frontend & backend origins match or adjust CORS(app) config to restrict allowed origins.
- **Port 5000 already in use**
  Change port in app.run(port=...) or kill the process using the port.
- **Permission errors on Linux for binding low ports**
  Use a port >1024 or set up reverse proxy (Nginx).

---

13 — Useful dev tips

- Keep your API key out of source control — use .env or system environment variables. Optionally use python-dotenv for local dev only.
- When experimenting with datasets, keep sensible naming for runs (RUN_NAME) so find_latest_weights() picks the correct run.
- For offline fallback of advice, implement a small rule-based fallback if Gemini fails.
- Add request logging and metrics for production (Prometheus, Grafana).